Finding Clusters using Node2Vec Embedding

May 2, 2025

Team Members

- **Daram Vishnuvardhan** CS24MTECH12002
- **Madhan V** CS24MTECH11021
- **P Rushikeshvar Reddy** CS24MTECH11018

1 Introduction

The objective of this assignment was to:

- Build a graph from the given `payments.xlsx` file where users are nodes and payments are edges.
- Merge multiple edges between the same pair of users to form a simple directed graph.
- Implement Node2Vec manually without using existing packages.
- Apply a clustering algorithm on node embeddings and visualize the clusters using PCA.

2 Implementation Details

2.1 Loading and Preparing the Data

The payments dataset was loaded using Pandas. Each row represented a payment from a source user to a destination user. A directed multigraph was created using `networkx.MultiDiGraph()` where nodes are users and edges are transactions.

2.2 Converting Multigraph to Simple Graph

Since there could be multiple payments between two users, all edges between a pair were merged:

- The number of payments between two users was counted.
- This count was assigned as the `weight` on the single edge in a `DiGraph`.

Thus, the resulting graph was a simple directed graph with aggregated edge weights.

2.3 Node2Vec Embedding (From Scratch)

- **Random Walk Generation**: For each node, multiple random walks were generated, treating nodes like words in sentences.
- **Skip-Gram Model with Negative Sampling**: A skip-gram model was implemented using PyTorch. Walks were used to train embeddings where nearby nodes in walks had similar representations.
- Training involved negative sampling and stochastic gradient descent optimization.

2.4 Clustering

After learning the node embeddings, **KMeans clustering** was applied. Nodes with similar embeddings were grouped into clusters.

2.5 Visualization

To better understand the structure of the learned embeddings, we performed two types of dimensionality reduction and visualizations:

PCA-based Visualization

Principal Component Analysis (PCA) was applied to reduce the high-dimensional embeddings to 2D. A scatter plot was created where each point represented a user, and colors indicated different clusters.

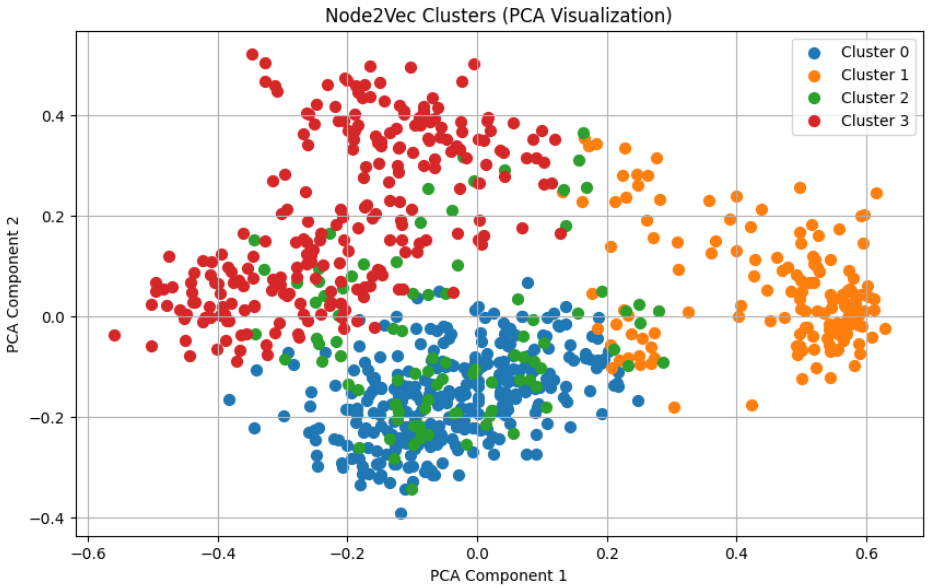


Figure 1: PCA-based Cluster Visualization of Node2Vec Embeddings

t-SNE-based Visualization

To further capture non-linear structures, t-Distributed Stochastic Neighbor Embedding (t-SNE) was used for dimensionality reduction. t-SNE often reveals more nuanced patterns than PCA, especially in high-dimensional data.

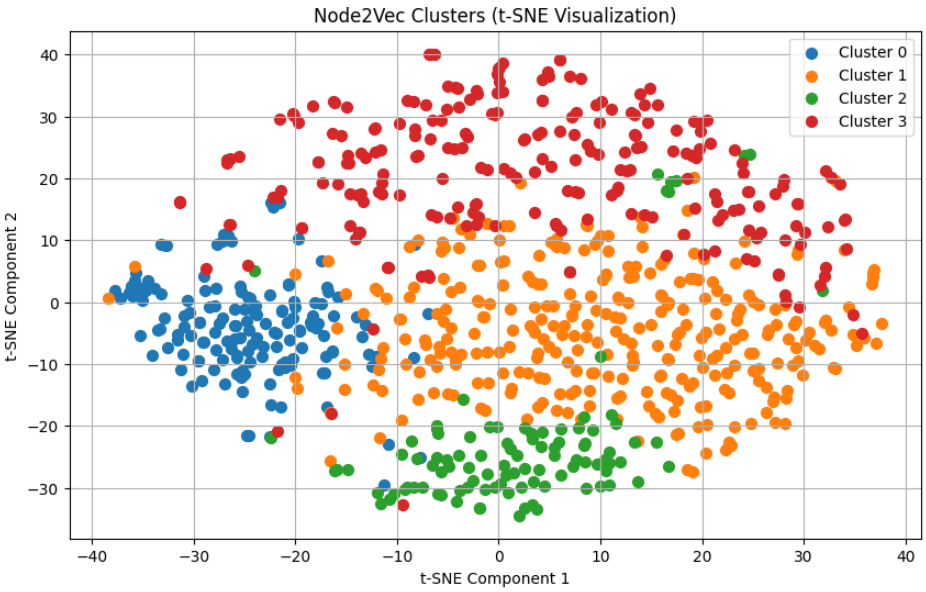


Figure 2: t-SNE-based Cluster Visualization of Node2Vec Embeddings

3 Results and Observations

- **Cluster Formation**: Both PCA and t-SNE visualizations show the formation of four clusters. However, the t-SNE plot shows clearer spatial separation and captures finer-grained differences between users.
- **Cluster 0 (Blue)**: Densely packed and isolated at the bottom-left, indicating a cohesive user group with similar transaction patterns.
- **Cluster 1 (Orange)**: Widely spread across the center region, suggesting high variance in user behavior. Some overlap with other clusters is visible.
- **Cluster 2 (Green)**: Located toward the bottom-center, showing a moderately compact structure but with some infiltration from adjacent clusters.
- **Cluster 3 (Red)**: Occupies the top and right regions, well-separated and consistent, pointing to another strong behavioral grouping.
- **Overlap and Spread**: Compared to PCA, t-SNE demonstrates more distinct cluster boundaries while still revealing natural overlaps—especially between Clusters 1 and 2. This reflects:
  1. *Behavioral Interactions*: Users interacting across communities may not belong strictly to a single group.
  2. *Embedding Sensitivity*: t-SNE retains local similarities more effectively than PCA, making it ideal for capturing subtle transitions between user behaviors.
- **t-SNE Reduction Quality**: Unlike PCA, t-SNE is non-linear and does not prioritize global variance. However, it excels at preserving local neighborhood structures, which is evident in the organic separation of clusters in the visualization.

4 Conclusion

- A simple directed graph was successfully built from the transactions data.

- Node2Vec was implemented from scratch, generating meaningful node embeddings.
- Clustering and visualization confirmed the ability of the embeddings to capture structural similarities among users.

5 Possible Improvements

- Hyperparameter tuning (walk length, window size, number of walks) could improve embedding quality.
- Using methods like the elbow method or silhouette score to determine the optimal number of clusters.